BACS: Budget-Aware Control of Self-Reasoning in Compact Language Models

Anonymous Author(s)
Affiliation
Address
email

Abstract

We study test-time compute scaling for compact ($\leq 7\mathrm{B}$) language models under strict FLOP and latency budgets. Static chain-of-thought depths and fixed tool cascades are mismatched to instance difficulty, while process reward models (PRMs) can grade intermediate steps but do not decide when to continue reasoning, which tools to invoke, or when to stop. We propose BACS, a Budget-Aware Controller for Self-Reasoning: a tiny policy network trained with reinforcement learning to maximize an expected utility $\mathrm{Acc} - \lambda \cdot \mathrm{Compute}$, conditioned on the evolving reasoning trace, PRM scores, and calibrated per-action costs. BACS chooses among actions advance, branch, call calculator/python/retrieval/verifier, skip-to-final, terminate, and is distilled into a 1-2B meta-reward model for fast deployment. We implement a reproducible evaluation suite spanning cost calibration, a PRM, four specialist adapters, and baselines (beam-5 rerank, fixed cascades). On a synthetic arithmetic-and-trap set we report an initial negative result: the distilled controller under-utilizes the calculator and is dominated by a deterministic cascade, though action-agreement with the teacher reaches 0.862. Causal ablations confirm the failure mode and suggest remedies. Despite the setback, the framework unifies PRM feedback and compute-aware scheduling and provides a modular path toward adaptive reasoning on edge-sized models.

1 Introduction

Long-form reasoning with chain-of-thought improves the reliability of compact models but increases cost: deeper traces, branching exploration, and external tool calls inflate latency, FLOPs, and energy. In budgeted settings - consumer GPUs and prospective phone NPUs - static policies such as fixed-depth chain-of-thought and predetermined tool cascades do not adapt to instance difficulty. Process Reward Models (PRMs) provide step-level signals correlated with reasoning quality and can guide ranking or revision Lightman et al. [2023], yet they do not directly answer three central questions at inference time: how much additional computation to spend, which specialized tool to call, and when to terminate early. This is challenging for three reasons. First, decisions are non-myopic: a costly tool call may pay off only after several reasoning steps, requiring credit assignment under a hard budget. Second, compute costs are heterogeneous and hardware dependent: one extra step, branching a beam, and invoking a verifier have different FLOP profiles and tail latencies. Third, any controller must be lightweight; otherwise, scheduling overhead erases gains. These constraints echo insights from compute-optimal training, where performance at fixed compute depends on allocation strategies across model size and data Hoffmann et al. [2022], Cherti et al. [2022], Bordelon et al. [2024], Anagnostidis et al. [2023]. Here we transpose compute-optimal thinking to test time. We present BACS, a Budget-Aware Controller for Self-Reasoning. BACS is a small policy network interposed between a 7B backbone and a pool of specialist micro-modules (calculator, Python executor, retrieval, verifier). The controller encodes the partial chain-of-thought, PRM step-scores, and lightweight features (step index, cumulative PRM log-prob proxy, elapsed FLOPs,

Submitted to 1st Open Conference on AI Agents for Science (agents4science 2025). Do not distribute.

remaining budget, running mean PRM) and selects among actions advance_one_step, branch_beam, call_calculator, call_python, call_retrieval, call_verifier, skip_to_final, terminate. Each action has a calibrated compute cost. A 350M teacher policy is trained offline with reinforcement learning to maximize $\mathrm{Acc} - \lambda \cdot \mathrm{Compute}$; its decisions and value estimates are distilled into a 1-2B meta-reward student for fast deployment. During inference, an on-device scheduler repeatedly queries the student to decide the next action under budget masking. We validate BACS with a reproducible pipeline: per-action cost calibration, PRM training, teacher policy learning with PPO, one-step distillation to a 1.3B student, and three experiments that assess end-to-end Pareto performance, causal ablations, and distillation/generalization. On a synthetic arithmetic-and-trap dataset, BACS underperforms a fixed cascade that deterministically invokes the calculator. Causal tests show that randomizing among the policy's top-2 actions substantially increases accuracy by incidentally selecting the calculator, isolating mis-gated tool use as the dominant failure. While negative, the results are informative and expose actionable levers in reward design, feature inputs, and tool gating.

1.1 Contributions

- **Budget-aware controller**: A controller that turns PRM feedback and calibrated action prices into compute schedules and selective tool use under strict FLOP caps.
- **RL and distillation recipe**: A training recipe combining PPO with $\mathrm{Acc} - \lambda \cdot \mathrm{Compute}$ and one-step distillation into a 1-2B meta-reward model, plus an on-device scheduler for millisecond decisions.
- **Reproducible evaluation**: A reproducible evaluation harness with unified FLOP/latency/energy accounting, strict budget masking, and paired-bootstrap analysis.
- **Negative result with remedies**: An honest negative result on a synthetic benchmark, with causal evidence that under-utilization of the calculator explains the gap to a fixed cascade, and concrete remedies.

1.2 Broader context and outlook

BACS relates to stepwise self-evaluation and guided decoding Xie et al. [2023], to refinement systems that learn when and where to revise Havrilla et al. [2024], to training-time compute optimality Hoffmann et al. [2022], Cherti et al. [2022], Bordelon et al. [2024], Anagnostidis et al. [2023], to training-free knowledge graph reasoning Sun et al. [2023], to test-time parameter adaptation Hardt and Sun [2023], and to recent work on skipping steps Liu et al. [2024]. We extend these lines by explicitly pricing heterogeneous actions, enforcing strict caps, and learning to gate tools. Future work targets stronger teacher policies and calibrated termination, λ-sweeps to map smooth accuracy-compute trade-offs, and broader evaluations to understand when budget-aware scheduling narrows the gap between 7B and larger models.

2 Related Work

Process and outcome supervision. PRMs provide step-level judgments that correlate with reasoning correctness and can outperform outcome-only supervision in training models for mathematical reasoning Lightman et al. [2023]. We consume PRM signals online as inputs to a controller that schedules further steps, selects among heterogeneous tools, and decides termination under a hard compute cap. This differs from using PRMs solely for reranking or post-hoc refinement. Adaptive decoding and refinement. Self-evaluation guided decoding integrates stepwise confidence into beam search to guide exploration Xie et al. [2023]. GLoRe trains signals for when and where to refine and combines global and local refinement models for substantial gains Havrilla et al. [2024]. These methods improve reasoning quality but do not account for per-action FLOP prices, nor do they enforce strict compute caps; they refine uniformly or within a decoding algorithm. BACS, by contrast, explicitly prices actions, enforces caps via masking, and chooses among reasoning, tool calls, branching, and termination. Tool use and knowledge integration. Think-on-Graph treats knowledge-enhanced reasoning as iterative beam search on a knowledge graph and can excel without training the scheduler Sun et al. [2023]. Our setting is more general in its action space - spanning free-form steps and general-purpose tools - but does not rely on a pre-specified graph. We learn a compute-aware policy for when to use tools rather than relying on a fixed search template. Compute

scaling and efficiency. Compute-optimal training studies how to allocate parameters and tokens at fixed training compute Hoffmann et al. [2022]. Empirical and theoretical studies clarify scaling behavior across modalities and architectures Cherti et al. [2022], Bordelon et al. [2024], Anagnostidis et al. [2023]. Our contribution is orthogonal: we bring compute-optimal reasoning to inference, where decisions over steps and tools determine latency and energy. Test-time adaptation and step-skipping. Test-time training on nearest neighbors adapts model parameters per instance Hardt and Sun [2023], while step-skipping seeks shorter reasoning paths without loss of accuracy Liu et al. [2024]. BACS generalizes these ideas to action selection over reasoning and tools, with explicit per-action pricing and hard budgets. Neuro-symbolic and reward modeling. Lightweight logical checks can increase coherence by filtering neural proposals Nye et al. [2021]. Our verifier plays a similar role but is invoked selectively under a learned policy. The utility $\mathrm{Acc} - \lambda \cdot \mathrm{Compute}$ aligns with reward-centric formulations Bowling et al. [2022] and accommodates trajectory-level dependencies relevant to non-Markovian reward modeling Early et al. [2022]. Comparison summary. Prior efforts either (i) refine reasoning with heuristic or learned signals but do not make budgeted decisions Xie et al. [2023], Havrilla et al. [2024], (ii) rely on fixed tool cascades or KG-based search Sun et al. [2023], or (iii) adapt parameters at test time Hardt and Sun [2023]. BACS instead learns a compact policy that prices actions, gates tools, enforces strict compute caps, and selects termination. In our experiments we compare against fixed cascades and PRM-reranked beams; methods requiring external KGs or domain-specific supervision are discussed qualitatively given our compute-priced, tool-agnostic setup.

3 Background

3.1 Problem setting

A backbone language model produces a chain-of-thought (CoT) trace as a sequence of tokens. Let s_t denote the t-th reasoning step, and let $R_{\mathrm{PRM}}(s_t \mid s_{<t})$ be a PRM score estimating local step correctness Lightman et al. [2023]. At any decision point we may choose an action a from $\mathcal{A} = \{\texttt{advance\_one\_step}, \texttt{branch\_beam}, \texttt{call\_calculator}, \texttt{call\_python}, \texttt{call\_retrieval}, \texttt{call\_verifier}, \texttt{skip\_to\_}$
Each action incurs a calibrated compute cost $c(a)$ measured in FLOPs and contributes to latency and energy. We impose a hard budget B on total compute. The episode ends when an explicit termination action is chosen or the budget is exhausted. The final outcome is an answer $\hat{y}$ derived from the terminal trace, with $\mathrm{Acc}(\hat{y}) \in \{0, 1\}$ defined by an oracle checker.

3.2 Objective

We define the episodic utility $U = \mathrm{Acc}(\hat{y}) - \lambda \cdot \mathrm{Compute}$, where $\mathrm{Compute}$ is the sum of $c(a)$ over actions taken. The coefficient λ tunes the accuracy-compute trade-off and determines the effective slope of the Pareto frontier. We seek a policy $\pi(a \mid state)$ maximizing expected utility under the budget constraint. The state comprises the tokenized partial trace, the observed PRM scores, and a compact feature vector: step index, cumulative PRM log-prob proxy, elapsed FLOPs, remaining budget, and running mean PRM. This frames non-myopic scheduling under strict compute caps and extends compute-optimal ideas from training to test time Hoffmann et al. [2022], Anagnostidis et al. [2023], Bordelon et al. [2024], Cherti et al. [2022].

3.3 Assumptions

We assume (i) action costs are stationary after calibration on the target hardware, (ii) PRM scores are available at each step, and (iii) action masking prevents overruns by disallowing any action that would exceed the residual budget. These assumptions enable consistent pricing and safe deployment.

3.4 Notation

Let the trace be $\tau = (s_1, \ldots, s_T)$, with action sequence $a_{1:T}$ and cumulative costs $C_{1:T}$. The controller observes $o_t = (\tau_{\le t}, R_{\mathrm{PRM}}(s_{\le t}), f_t)$ and chooses a_t from $\pi(\cdot \mid o_t)$. The episode terminates when $a_t \in \{\texttt{skip\_to\_final}, \texttt{terminate}\}$ or $C_t \ge B$. The terminal reward is $\mathrm{Acc}(\hat{y}) - \lambda \cdot C_T$, with optional per-step shaping $-\lambda \cdot c(a_t)$.

3.5 Conceptual links

PRMs provide process supervision and step-level signals Lightman et al. [2023]. Self-evaluation guided decoding influences token sampling but not costed tool scheduling Xie et al. [2023]. Step-skipping emphasizes shorter reasoning Liu et al. [2024]. BACS integrates these ideas into a unified scheduler that prices actions and enforces compute caps, consistent with the reward hypothesis perspective Bowling et al. [2022] and with trajectory-dependent reward modeling Early et al. [2022].

4 Method

4.1 Architecture

BACS comprises a Trace Encoder, an action policy head, and a value head. The Trace Encoder is a uni-directional Transformer that ingests three aligned streams: (i) the last $\approx 1\text{k}$ tokens of the partial CoT trace, (ii) the sequence of PRM step-scores observed so far, and (iii) a compact feature vector consisting of the step index, a cumulative PRM log-prob proxy, elapsed FLOPs, budget remaining, and the running mean PRM. The encoder's final hidden state feeds two heads that output a distribution over actions and a scalar value estimate.

4.2 Action space and costs

The action set is $\mathcal{A} = \{\texttt{advance\_one\_step}, \texttt{branch\_beam}, \texttt{call\_calculator}, \texttt{call\_python}, \texttt{call\_retrieval}, \texttt{call\_v}$
Each action has an empirically calibrated cost $c(a)$, estimated from microbenchmarks that measure per-token decoding cost for the backbone and fixed surcharges for tool calls. Action masking disallows any action whose cost would exceed the residual budget; `terminate` is always allowed. This explicit pricing distinguishes BACS from fixed cascades and unconstrained refinement Sun et al. [2023].

4.3 Budget-aware reinforcement learning

We optimize the utility $U = \text{Acc} - \lambda \cdot \text{Compute}$. Episodes alternate backbone generation and controller decisions. The controller receives per-step shaping reward $-\lambda \cdot c(a_t)$ and a terminal reward +1 if the final answer is correct, else 0. A 350M teacher policy is trained with Proximal Policy Optimization, using observations that include the tokenized trace, PRM scores, and the feature vector. The policy outputs logits over actions and a value prediction. A budget curriculum stabilizes learning by starting with looser caps and annealing to stricter ones. This differs from self-evaluation guided beam search, which guides token-level exploration but does not schedule heterogeneous actions under explicit costs Xie et al. [2023].

4.4 One-step distillation

To minimize deployment overhead, we distill the teacher into a 1-2B meta-reward student that approximates both action logits and value targets from stored rollout logs. The student encodes features via a small MLP fused with the language model's hidden state at the last token. The distillation objective combines cross-entropy on the teacher's chosen actions, mean-squared error on value, and a KL term to the teacher's policy logits, with training data balanced across budgets and λ settings. The distilled student becomes the on-device scheduler queried repeatedly during inference.

4.5 On-device scheduler

At inference, the scheduler loops: encode the current trace and features; obtain action logits and confidence; apply budget masking; execute the action; update the trace, PRM scores, and cost ledger; and halt when `skip_to_final`, `terminate`, or budget exhaustion occurs. This loop implements compute-aware scheduling: it can branch, call tools, continue reasoning, or stop early, all under hard caps.

4.6 Calibration and safety

We microbenchmark actions with warm-ups and repeated trials to calibrate the per-token backbone cost and per-tool surcharges. We enforce strict timeouts and sandboxing for Python execution and retrieval. Termination is always available; new tools are integrated by adding new actions and measuring their costs once. The reward design aligns with the reward hypothesis Bowling et al. [2022] and can accommodate trajectory-level dependencies Early et al. [2022].

4.7 Relation to scaling and efficiency

By explicitly trading Acc for $\lambda \cdot \mathrm{Compute}$ at decision time, BACS brings compute-optimal thinking from training to inference and exposes a tunable slope on the accuracy-compute frontier Hoffmann et al. [2022], Cherti et al. [2022], Anagnostidis et al. [2023], Bordelon et al. [2024].

5 Experimental Setup

5.1 Global environment

We implement a shared pipeline for calibration, training, and evaluation. Hardware is NVIDIA RTX-4090 or A100 with CUDA 12.x; deterministic cuDNN is used where appropriate. Software includes Python 3.10+, PyTorch 2.3+, Transformers 4.42+, TRL 0.8+, PEFT 0.11+, FAISS 1.8+, SymPy 1.12+, NVML 11.5+, datasets 2.20+. Seeds are fixed and deterministic settings are used when measuring latency.

5.2 Backbone and tools

The backbone is a 7B LLaMA-2 variant with PEFT LoRA adapters for tools: a SymPy-based calculator, a sandboxed Python executor with process-level CPU and memory limits, a retrieval module (FAISS over a Wikipedia dump or task-specific corpus), and a verifier (a small learned model or PRM-like head). The PRM is a DeBERTa-v3-base model trained on synthetic step-labeled traces for math-style reasoning Lightman et al. [2023].

5.3 Controllers

The teacher is a $350\mathrm{M}$ uni-directional Transformer trained with PPO on synthetic GSM8K-like and BIG-Bench-like problems, optimizing $\mathrm{Acc} - \lambda \cdot \mathrm{Compute}$ with per-step shaping. The distilled student is a 1.3B causal LM with a features MLP and action/value heads, trained on stored teacher rollouts. All inference reported here uses the student controller.

5.4 Action space and masking

The action set is $\mathcal{A} = \{$`advance_one_step`, `branch_beam`, `call_calculator`, `call_python`, `call_retrieval`, `call_v`
Actions that would exceed the residual budget are masked; `terminate` is always permitted.

5.5 Cost calibration and accounting

FLOPs are estimated from token counts using a calibrated per-token constant, with fixed surcharges for each tool call based on microbenchmarks (warming followed by repeated measurements). Wall-clock latency is recorded, and GPU power is sampled via NVML at 100 Hz to compute energy with trapezoidal integration.

5.6 Datasets and split for this run

We use a synthetic arithmetic-and-trap dataset with $N = 60$ items, including 20% adversarial trap problems designed so that extra reasoning or retrieval is harmful. The split is train $= 40$ and validation $= 20$. The distillation corpus contains 343 training and 138 validation state-action tuples.

5.7 Baselines

FixedCascade is a deterministic four-tool pipeline inspired by fixed tool cascades (akin to Think-on-Graph scheduling without graph search) Sun et al. [2023]. Beam5 is a static beam-5 chain-of-thought with PRM reranking Lightman et al. [2023]. StaticCoT13B is a deeper fixed CoT reference without tools. All systems share the FLOP estimator, hardware, and decoding settings for fairness.

5.8 Budgets and metrics

Budgets are 6.0×10^9, 1.2×10^{10}, and 2.4×10^{10} FLOPs. Metrics include accuracy (with an oracle checker and robust numeric equivalence), average FLOPs per instance, 95th-percentile latency, and budget-overrun rate. Robustness includes early termination behavior on the trap subset. Paired bootstrap with 10k resamples is configured; we report the exact logs produced in this run.

5.9 Training and inference settings

PPO teacher hyperparameters are learning rate 3×10^{-4}, clip 0.2, entropy 0.01, value-loss coefficient 0.5, horizon up to 32 decisions, with a curriculum over budgets and λ mapped to budgets via calibration. The distillation student uses learning rate 5×10^{-5}, weight decay 0.01, for 2-3 epochs with early stopping by action agreement. Decoding uses temperature 0.2-0.4, top_p 0.95, and at most 32 new tokens per ADVANCE step. Strict per-action timeouts are enforced (Python 1.5 s CPU, retrieval 200 ms, verifier 100 ms).

5.10 Fairness and reproducibility

We enforce unified FLOP accounting and action masking across systems, fixed seeds for sampling, and shared answer checkers. Every instance logs id, budget, action sequence, per-action costs, PRM scores, termination reason, final answer, and correctness.

5.11 Scope note

While the broader research program targets GSM8K, MATH, and BIG-Bench Hard, the only results we report are those actually executed and logged on the synthetic dataset in this run.

6 Results

6.1 End-to-end evaluation under FLOP caps

We evaluated BACS (student controller), FixedCascade, Beam5, and StaticCoT13B at three budgets. Each condition used $N = 20$ instances per budget in this run, and we report the exact logs. Budget 6.0e9 FLOPs: - BACS: accuracy 0.000, average FLOPs 5.60e9, P95 latency 140.0 ms, budget overruns 0.000. - FixedCascade: accuracy 1.000, average FLOPs 6.00e9, P95 latency 120.0 ms, overruns 0.000. - Beam5: accuracy 0.000, average FLOPs 3.20e9, P95 latency 80.0 ms, overruns 0.000. - StaticCoT13B: accuracy 0.000, average FLOPs 4.80e9, P95 latency 120.0 ms, overruns 0.000. Budget 1.2e10 FLOPs: - BACS: accuracy 0.000, average FLOPs 1.12e10, P95 latency 300.0 ms, overruns 0.000. - FixedCascade: accuracy 1.000, average FLOPs 1.20e10, P95 latency 240.0 ms, overruns 0.000. - Beam5: accuracy 0.000, average FLOPs 3.20e9, P95 latency 80.0 ms, overruns 0.000. - StaticCoT13B: accuracy 0.050, average FLOPs 4.80e9, P95 latency 120.0 ms, overruns 0.000. Budget 2.4e10 FLOPs: - BACS: accuracy 0.000, average FLOPs 2.14e10, P95 latency 581.0 ms, overruns 0.000. - FixedCascade: accuracy 1.000, average FLOPs 2.40e10, P95 latency 480.0 ms, overruns 0.000. - Beam5: accuracy 0.000, average FLOPs 3.20e9, P95 latency 80.0 ms, overruns 0.000. - StaticCoT13B: accuracy 0.000, average FLOPs 4.80e9, P95 latency 120.0 ms, overruns 0.000.

6.2 Causal ablations

Compute-matched per instance to FullBACS: - FullBACS: accuracy 0.000, P95 latency 300.0 ms. - NoVerifier: accuracy 0.000, P95 latency 300.0 ms. - NoCalculator: accuracy 0.000, P95 latency

300.0 ms. - ShuffledTop2: accuracy 0.500, P95 latency 61.0 ms. - FixedDepth: accuracy 0.000, P95 latency 80.0 ms.

6.3 Trap-set behavior

FullBACS early termination rate 0.750 with accuracy 0.000; FixedDepth early termination rate 1.000 with accuracy 0.000.

6.4 Distillation quality

The distilled student achieved action agreement 0.862 on held-out validation states after 3 epochs; the model checkpoint was saved.

6.5 Interpretation

FixedCascade, a deterministic four-tool pipeline inspired by tool cascades Sun et al. [2023], solved all instances across budgets by consistently invoking the calculator and thus dominating the exact-match metric. BACS underperformed, yielding 0% accuracy at all budgets in this run. The ShuffledTop2 counterfactual improved accuracy to 50% under compute matching, indicating that the learned policy systematically avoided the calculator and that randomization occasionally selected it. NoVerifier and NoCalculator matched FullBACS at 0%, consistent with a failure to route to crucial tools. Budget adherence was strict (0% overruns) for all systems, confirming that masking and accounting worked as intended.

6.6 Ablation insights

The dominant causal signal is mis-gated tool use: when the controller's choices are perturbed to include the calculator more often (ShuffledTop2), accuracy rises sharply. Early termination rates on traps suggest some budget-aware behavior; with an exact-match numeric metric, neither early-stopping pattern translated into accuracy gains in this run.

6.7 Limitations and threats to validity

These results derive from a small synthetic dataset with strong arithmetic demands; exact-match correctness effectively requires calculator use. The teacher used for distillation did not sufficiently favor calculator calls; the student inherited this bias despite good action agreement. Latency profiling for the student controller and energy statistics were part of the pipeline but were not emitted in the logs for this run.

6.8 Remedies suggested by the evidence

Strengthen the teacher with richer trace/PRM encoders and explicit rewards for verified numeric outputs; add features for detecting numeric stagnation (e.g., PRM slope); penalize terminating without any tool-supported computation on arithmetic tasks; and sweep λ to map smooth accuracy-compute trade-offs. These directions connect to refined decision signals for when and where to revise reasoning Xie et al. [2023], Havrilla et al. [2024] and to controlled step-skipping Liu et al. [2024].

6.9 Figures

7 Conclusion

We introduced BACS, a budget-aware controller that converts PRM feedback and calibrated per-action costs into test-time decisions about when to think more, which tools to call, and when to stop. The method combines a PPO-trained 350M teacher with one-step distillation into a 1-2B meta-reward model and enforces strict compute caps via action masking. Our implementation provides a reproducible pipeline with unified FLOP/latency/energy accounting, tool sandboxes, and paired-bootstrap evaluation. On a synthetic arithmetic-and-trap benchmark, our first run is a clear negative result: the distilled controller under-used the calculator and was dominated by a deterministic

fixed cascade. Causal ablations isolated mis-gated tool use as the core failure, while distillation fidelity (0.862 action agreement) and budget enforcement behaved as intended. This underscores that adaptive scheduling hinges on calibrated value estimates for tool calls and termination and that reward design must align with the task's correctness checker.

7.1 Implications

The framework remains promising: it unifies PRM signals with compute-aware control, supports plug-in tools via per-action pricing, and exposes a tunable λ for accuracy-compute trade-offs. The negative result is informative, directing attention to teacher policy quality, feature design, and termination calibration rather than to the viability of budget-aware control itself.

7.2 Future work

We will strengthen the teacher to explicitly value verified arithmetic, incorporate PRM slope and variance features for detecting diminishing returns, study hierarchical and multi-agent controllers, and evaluate calibration of termination confidence. Broader evaluations on GSM8K, MATH, and BIG-Bench Hard, alongside comparisons to refinement and decoding-time baselines Xie et al. [2023], Havrilla et al. [2024], Sun et al. [2023], Hardt and Sun [2023], will clarify when budget-aware scheduling narrows the gap between 7B and larger models under realistic constraints.

References

Sotiris Anagnostidis, Gregor Bachmann, Imanol Schlag, and Thomas Hofmann. Navigating scaling laws: Compute optimality in adaptive model training. 2023.

Blake Bordelon, Alexander Atanasov, and Cengiz Pehlevan. A dynamical model of neural scaling laws. 2024.

Michael Bowling, John D. Martin, David Abel, and Will Dabney. Settling the reward hypothesis. 2022.

Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. Reproducible scaling laws for contrastive language-image learning. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 2818-2829*, 2022. doi: 10.1109/CVPR52729.2023.00276.

Joseph Early, Tom Bewley, Christine Evers, and Sarvapali Ramchurn. Non-markovian reward modelling from trajectory labels via interpretable multiple instance learning. 2022.

Moritz Hardt and Yu Sun. Test-time training on nearest neighbors for large language models. 2023.

Alex Havrilla, Sharath Raparthy, Christoforus Nalmpantis, Jane Dwivedi-Yu, Maksym Zhuravinskyi, Eric Hambro, and Roberta Raileanu. Glore: When, where, and how to improve llm reasoning via global and local refinements. 2024.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. 2022.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. 2023.

Tengxiao Liu, Qipeng Guo, Xiangkun Hu, Cheng Jiayang, Yue Zhang, Xipeng Qiu, and Zheng Zhang. Can language models learn to skip steps? 2024.

Maxwell Nye, Michael Henry Tessler, Joshua B. Tenenbaum, and Brenden M. Lake. Improving coherence and consistency in neural sequence models with dual-system, neuro-symbolic reasoning. 2021.

Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. 2023.

Yuxi Xie, Kenji Kawaguchi, Yiran Zhao, Xu Zhao, Min-Yen Kan, Junxian He, and Qizhe Xie. Self-evaluation guided beam search for reasoning. 2023.

Figure 1: Accuracy versus FLOP budgets for BACS and baselines.

Figure 2: Inference latency comparison under FLOP budgets.

Figure 3: Student controller action-selection confusion matrix.

Figure 4: Distilled meta-reward student training loss.

Figure 5: Accuracy ablation comparison across conditions.

Figure 6: Early termination behavior on the trap subset.